

The Mathematics of Processing-Aware Communication

The Three Costs of Every Message

Registry: [\[HOWL-MATH-19-2026\]](#)

Series Path: [\[HOWL-INFO-11-2026\]](#) → [\[HOWL-INFO-12-2026\]](#) → [\[HOWL-INFO-13-2026\]](#) → [\[HOWL-MATH-14-2026\]](#) → [\[HOWL-MATH-15-2026\]](#) → [\[HOWL-MATH-16-2026\]](#) → [\[HOWL-MATH-17-2026\]](#) → [\[HOWL-MATH-18-2026\]](#) → [\[HOWL-MATH-19-2026\]](#)

Date: June 2026

DOI: 10.5281/zenodo.20630502

Domain: Information Processing Theory / Applied Mathematics

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude Opus 4.6.

1. The Problem Shannon Solved and the One He Didn't

The email arrived. The documentation page loaded. The Slack message appeared. The lecture's audio was clear. The bits traversed the channel and arrived intact. Shannon's theory guarantees this — given sufficient encoding, any message can be transmitted reliably through a noisy channel. Modern channels approach this guarantee routinely. Messages arrive.

And yet communication fails. A junior developer reads the senior architect's design document three times and cannot connect the caching strategy to anything they know. A patient hears the physician's explanation of their diagnosis and walks away confused. A student reads the textbook paragraph about eigenvalues and finds it impenetrable despite every word being spelled correctly. A new hire reads the onboarding wiki and finds it simultaneously too detailed in some areas and too sparse in others.

The channel worked. The bits arrived. The failure lives at the endpoints — in the cost of transforming thoughts into symbols at the sender, and especially in the cost of transforming symbols into understanding at the receiver. These costs exist outside Shannon's framework. His 1948 formalization drew a clean boundary: source produces symbols, encoder transforms them for the channel, channel transmits with noise, decoder recovers symbols, destination receives. He formalized the middle three stages with full mathematical rigor. What the source does before producing symbols and what the destination does after receiving them, he explicitly excluded.

This paper formalizes the excluded territory — the endpoint costs — and shows that for most real-world communication, they dominate the channel cost Shannon formalized. The channel is not where communication fails. The endpoints are. Optimizing the channel is necessary but insufficient. Optimizing the endpoints requires reasoning about what the

sender and receiver have dissolved and what they haven't, which requires a theory of processing that Shannon declared out of scope.

The vocabulary builds in order. Processing is what any system does when it must act on information — a CPU executing instructions, a physician diagnosing, a developer debugging, a student learning. The unit of processing is the **op**: one irreducible transformation by one processor. Processing entropy is the op count a specific processor requires for a specific task — it is receiver-dependent, unlike Shannon's entropy which is a property of the source regardless of who receives the message. When processing entropy reaches zero through repeated engagement under consistent conditions, the task is **dissolved**: handled structurally without consuming the processor's scarce sequential pipeline. The processor's capacity is bounded by one inequality: total ops multiplied by average op duration must not exceed the available time budget.

2. The Three-Term Cost Equation

Every act of communication has three costs. They are independent, additive, and measured in different units. Together they determine the total price of getting information from one processor to another.

Sender encoding cost: $H_p(A, \text{encode})$. The ops the sender executes to transform their internal state into transmissible symbols. An expert explaining a dissolved concept incurs near-zero encoding cost — the words come without effort, the structure is automatic, the vocabulary selection requires no deliberation. A novice trying to explain something they barely understand incurs high encoding cost — they must actively search for words, simplify their fuzzy understanding into precise statements, reformulate when their first attempt doesn't capture the meaning. The encoding cost is determined by the sender's dissolution state for both the content and the act of explaining it.

Channel cost: $H_s(\text{channel})$. The bits required for reliable transmission. This is Shannon's domain — fully formalized, with exact theorems about minimum encoding rates, channel capacity, and error correction. For text: roughly the character count times the encoding scheme's bits per character. For speech: bandwidth times duration. For a network packet: payload plus protocol overhead plus error correction. The channel cost depends on the message's statistical properties and the channel's noise characteristics. It does not depend on who is sending or receiving. A sentence costs the same number of bits whether an expert or a novice wrote it, and whether an expert or a novice reads it.

Receiver decoding cost: $H_p(B, \text{decode})$. The ops the receiver executes to transform received symbols into actionable understanding. This is the critical term — the one Shannon excluded and the one that dominates most real communication. An experienced physician reads a radiology report and extracts the diagnosis in one or two ops: the terminology is dissolved, the report structure is dissolved, the diagnostic implications fire automatically. A medical student reads the same report and requires twenty to thirty ops: each technical term must be consciously looked up or reasoned through, the report structure must be actively parsed, and the diagnostic implications must be explicitly constructed from first

principles.

The total cost:

$$\text{Cost}(A \rightarrow B) = \text{Hp}(A, \text{encode}) + \text{Hs}(\text{channel}) + \text{Hp}(B, \text{decode})$$

Each term is independent. The sender's encoding cost does not affect the channel cost (a carefully crafted sentence and a hastily typed one may require the same bits). The channel cost does not affect the receiver's decoding cost (a message arriving over fiber optic or carrier pigeon costs the receiver the same ops to understand, assuming identical content). The receiver's decoding cost does not affect the sender's encoding cost (the sender pays their encoding ops whether or not the receiver understands).

Shannon's framework is recovered as the special case where both endpoint terms are zero. When the sender has dissolved both the content and the encoding process, $\text{Hp}(A, \text{encode}) \approx 0$. When the receiver has dissolved both the vocabulary and the content domain, $\text{Hp}(B, \text{decode}) \approx 0$. The only remaining cost is $\text{Hs}(\text{channel})$ — exactly what Shannon formalized. Expert-to-expert communication in a shared domain with shared vocabulary approaches this case. Two experienced surgeons discussing a procedure they've both performed hundreds of times. Two senior developers discussing a codebase they both maintain. Two mathematicians discussing a theorem they both know. The words are free at both ends. The only cost is moving them through space.

For most real-world communication, the picture is different. Modern channels are cheap — email costs fractions of a cent, web pages load in milliseconds, voice travels in real time. The channel term Hs is small and well-optimized. The processing terms are large. A developer spending forty-five minutes wrestling with an API they've never used is paying in ops, not bits. A student spending three hours on a textbook chapter is paying in ops. The total cost of the communication is dominated by the receiver's processing entropy, and Shannon's framework — which optimizes the cheapest of the three terms — is necessary for the bits to arrive but insufficient for understanding to occur.

3. Compression and Decompression

Language works by compression. A single word packs many referents into one transmissible token. The word “fire” compresses combustion, employment termination, weapon discharge, ceramic kiln process, artistic inspiration, and dozens of other referents into a single syllable. The technical term “eigenvalue” compresses the scalar λ such that $A\mathbf{v} = \lambda\mathbf{v}$, with its implications for matrix diagonalization, spectral decomposition, stability analysis, and principal component analysis, into a single word. Each token is a compression function: many referents in, one symbol out.

Define the **compression function** C and its inverse:

C : referent_space $\rightarrow$ token

C^{-1} : (token, context) $\rightarrow$ referent

Compression is many-to-one — many referents map to one token. Decompression is context-dependent — the same token decompresses to different referents depending on context. “Fire” in a burning building decompresses to combustion. “Fire” in a boardroom decompresses to termination. “Fire” on a rifle range decompresses to weapon discharge. Context resolves the ambiguity. But resolution is only possible if the receiver has dissolved the context-dependent decompression rules — if the receiver knows, without conscious effort, which referent “fire” maps to in each context.

Define the **compression ratio** of a token for a given processor:

$\text{ratio}(\text{token}, \text{processor}) = |\{\text{referents processor can decompress from token across all contexts}\}|$

A child’s compression ratio for “fire” might be three — the hot thing, the fire truck, the command to stop doing something. An adult’s might be fifteen. A firefighter’s might be fifty — including structural collapse indicators, flashover signs, ventilation patterns, incident command protocols. An arson investigator’s might be a hundred — adding accelerant signatures, burn pattern analysis, origin determination methods, chain of custody procedures, legal standards of evidence. The compression ratio grows with experience because experience dissolves more referent-to-token associations.

Define the **decompression cost** of a token:

$\text{decomp}(\text{token}, \text{processor}, \text{context}) = \text{Hp}(\text{processor}, \text{decode}(\text{token}, \text{context}))$

When the token is dissolved for the processor — the word is in their native vocabulary, the technical term is fluent, the context-dependent referent selection fires without deliberation — the decompression cost is zero. When the token is unfamiliar, the decompression cost is positive: the processor must actively decode it, which consumes ops from the scarce pipeline.

This is what makes language possible at conversational speed. Fluent speakers have dissolved their common vocabulary to zero decompression cost. Each word arrives and decompresses automatically — no pipeline allocation required. A typical speaker processes 150 words per minute in conversation. At zero decompression cost per common word, the pipeline is free to focus on meaning, nuance, and response formulation. If each word cost even one op of conscious decoding, the pipeline would be saturated by the incoming word stream before any higher-level processing could occur. Fluent communication presupposes dissolved decompression. Every word that is not dissolved is a processing tax on the message it appears in.

4. The Dissolution Differential

The three-term cost equation reveals a structural asymmetry between sender and receiver that explains why expert communication so often fails.

The expert sender has dissolved their domain. Technical terms cost zero encoding ops — the words come as naturally as breathing. The expert has also dissolved the act of explana-

tion within their peer group — they know what to say to another expert and the encoding is effortless. Their encoding cost $H_p(A, \text{encode})$ is near zero for domain communication.

But the expert's intuition about what words cost the receiver is calibrated to their own dissolution state. Every technical term is free for the expert. The expert underestimates the receiver's cost for those terms because the expert cannot easily re-experience what it was like before dissolution. This is not a failure of empathy — it is a structural consequence of dissolution. A dissolved skill is no longer visible to introspection. The expert literally cannot feel the cost of a term they have dissolved, and therefore cannot feel the cost that same term imposes on someone who hasn't.

Define the **dissolution differential** between sender and receiver for a token set:

$$\Delta(A, B, \text{tokens}) = \sum_t [H_p(B, \text{decode}(t)) - H_p(A, \text{decode}(t))]$$

When both have identical dissolution states, the differential is zero — every token costs each of them the same. When the sender is expert and the receiver is novice, the differential is large and positive — the receiver pays far more than the sender would for every undissolved token.

The dissolution differential is the quantitative measure of the communication gap. Consider a design document written by a senior architect. The document contains fifty technical terms — API names, architectural patterns, infrastructure concepts, team-specific jargon. For the architect, each term has zero decompression cost: dissolved through years of use. For a new team member, perhaps thirty of the fifty terms are unfamiliar, each requiring five to ten ops to decode — reading the term, attempting to recall its meaning, failing, searching documentation, reading the definition, connecting it to context, and resuming the sentence. The dissolution differential for this document: thirty terms times five to ten ops each, or 150 to 300 ops of additional receiver cost that the sender did not anticipate because, from the sender's perspective, every term was free.

The differential predicts communication difficulty before the message is sent. If the sender can estimate which tokens are dissolved for the receiver and which aren't, the differential is computable. The tokens contributing most to the differential are the specific points where communication will break down — the exact terms or concepts where the sender has zero cost and the receiver has high cost.

The intervention follows: replace high-differential tokens with lower-differential alternatives (use simpler words), or add dissolution infrastructure (definitions, examples, analogies) that reduces the receiver's decompression cost for the high-differential tokens. Both strategies increase H_s (the message gets longer or uses more common vocabulary, which has lower compression ratio). The increase in H_s is the price of reducing $H_p(B, \text{decode})$. Whether the tradeoff is worthwhile depends on the relative magnitudes — and for high dissolution differentials, it almost always is.

5. Redundancy as Dissolution Infrastructure

Shannon demonstrated that redundancy in the channel enables error correction — extra bits help recover from noise-induced corruption. This paper shows a parallel role for redundancy in processing: extra words in the message help reduce the receiver's decompression cost.

When a sender adds explanatory context to a message — defining a term, providing an example, restating a concept in simpler language, offering an analogy — they increase the channel cost H_s . Each additional word is bits on the channel. From Shannon's perspective, these words are redundant: they don't add information content that the message doesn't already carry. The expert reader gains nothing from them.

From the processing perspective, they are not redundant at all. They are **dissolution infrastructure**: structural support that provides, at channel cost, the decompressed associations that the expert receiver already has at zero cost but that the novice receiver would need to construct from scratch at high op cost.

Define **explanatory redundancy**:

$$\text{redundancy}(\text{message}) = H_s(\text{message}) - H_s(\text{minimal_encoding})$$

where `minimal_encoding` is the shortest encoding that preserves all information content — the Shannon-optimal version. Every word beyond the minimal encoding is, in Shannon's framework, waste. In the processing framework, each such word has a measurable effect on receiver cost.

Define the **dissolution efficiency** of an additional word:

$$\eta(\text{word}, \text{receiver}) = -\Delta H_p(B, \text{decode}) / \Delta H_s$$

The numerator is the reduction in receiver decoding cost caused by including the word. The denominator is the channel cost of the word. If including a one-word definition reduces a novice receiver's decoding cost by six ops, the dissolution efficiency is six — the word pays for itself six times over in receiver cost savings.

Not all redundancy is efficient. A word that restates something the receiver already understands has zero dissolution efficiency — it adds channel cost without reducing processing cost. A word that introduces new confusion has negative efficiency — it increases both channel and processing cost. The art of clear writing, formalized: every word should have positive dissolution efficiency for the target receiver population. Words with zero or negative efficiency should be removed.

Shannon-optimal encoding minimizes H_s alone. It produces maximally compressed messages — all jargon, no explanation, minimum word count. This is optimal when both endpoint terms are zero (expert-to-expert with shared vocabulary). **Processing-optimal encoding** minimizes the total:

$$\text{encoding}^* = \text{argmin}_e \{ H_p(A, \text{encode}(e)) + H_s(e) + H_p(B, \text{decode}(e)) \}$$

When the receiver's processing entropy is high, the processing-optimal message is longer than the Shannon-optimal message. The additional words reduce $H_p(B, \text{decode})$ by more

than they increase H_s . The optimal message length is where the marginal dissolution efficiency equals one — where one more word of explanation would cost exactly as much in channel capacity as it saves in receiver processing.

This produces a formal result: **the optimal message length increases with the dissolution differential**. The larger the gap between sender and receiver dissolution states, the more dissolution infrastructure the message must carry, and the longer it must be. This is not verbosity. It is optimal encoding for a receiver whose decompression costs are high.

6. The Optimization Surface

The three-term cost defines a surface over the space of possible encodings. For a fixed content (what the sender wants to communicate), the sender chooses an encoding — specific words, level of detail, structure, amount of explanation. Each choice determines all three cost terms.

The surface has characteristic regions.

The expert-shorthand minimum. The sender uses maximally compressed encoding — all jargon, no explanation, minimum words. Channel cost is minimal. Sender encoding cost is minimal (dissolved vocabulary, no effort required). Receiver cost depends entirely on the receiver's dissolution state: zero for an expert peer, extremely high for a novice. This encoding minimizes sender cost and channel cost at the expense of receiver cost. It is optimal if and only if the receiver is also an expert in the same domain with the same vocabulary.

The verbose-tutorial maximum. The sender explains everything from first principles — every term defined, every concept built from simpler concepts, extensive examples, multiple restatements. Channel cost is high. Sender encoding cost is moderate to high (constructing thorough explanations is work even for experts). Receiver cost is low for novices (ample dissolution infrastructure) but the total cost is high because the channel term is large. This encoding minimizes receiver cost at the expense of channel cost.

The processing-optimal saddle. Between these extremes lies the encoding that minimizes total cost. It uses dissolved vocabulary where the receiver has dissolved it (no unnecessary explanation), provides dissolution infrastructure where the receiver needs it (targeted definitions and examples), and omits explanation where the channel cost exceeds the processing benefit (no explanation of concepts the receiver cannot yet absorb).

The optimal encoding depends on both endpoints. It is not a property of the content alone, nor of the sender alone, nor of the receiver alone. It is a property of the sender-receiver pair and their respective dissolution states. This is why the same content requires different optimal encodings for different audiences — not as a stylistic choice but as a mathematical consequence of the cost equation.

For a given sender-receiver pair with known dissolution states, the optimal encoding is the one that minimizes:

$$H_p(A, \text{encode}(e)) + H_s(e) + H_p(B, \text{decode}(e))$$

The sender encoding cost typically varies slowly across encodings — experts can produce most encodings at similar cost, novices find all encoding costly. The channel cost varies linearly with message length. The receiver decoding cost varies dramatically with encoding choice — the right explanation can reduce it by orders of magnitude, the wrong explanation can increase it.

The optimization surface has a gradient that points the way to improvement. For any current encoding, the gradient indicates whether adding words (increasing H_s to decrease $H_p(B)$) or removing words (decreasing H_s while accepting increased $H_p(B)$) improves total cost. Following the gradient from any starting encoding leads to the total-cost minimum for that sender-receiver pair.

7. The Heterogeneous Audience

The optimization changes fundamentally when the sender addresses multiple receivers with different dissolution states.

A teacher addresses a classroom where some students have dissolved prerequisite concepts and others have not. A documentation page is read by experts who need a quick reference and by novices who need a tutorial. A meeting includes engineers, product managers, and designers with different dissolved vocabularies. An API serves developers ranging from first-day novices to decade-long veterans.

For N receivers with dissolution states $B_1, B_2, \dots, B_n$, the total cost of a single message is:

$$\text{Cost}_{\text{total}} = H_p(A, \text{encode}) + H_s + \sum_{i=1}^N H_p(B_i, \text{decode})$$

The sender encoding cost and channel cost appear once — one message, one encoding, one transmission. The receiver cost is summed across all receivers because each independently decodes the same message.

The receiver sum changes the optimization's character. Adding one word of dissolution infrastructure (defining a term, providing an example) costs ΔH_s once on the channel. It benefits every receiver who hasn't dissolved that concept. If k of the N receivers need the explanation, the benefit is $k \times \Delta H_p$ — the per-receiver cost reduction times the number of receivers who benefit. The cost-benefit ratio for the added word scales with k : the more receivers who need it, the more valuable the dissolution infrastructure.

Define the **audience dissolution profile** as the distribution of dissolution states across receivers for the relevant token set. High heterogeneity means the dissolution states span a wide range — some receivers have dissolved most tokens, others have dissolved few. Low heterogeneity means receivers cluster at similar dissolution states.

Define the **audience-weighted dissolution efficiency** of an additional word:

$$\eta_{\text{audience}}(\text{word}) = \sum_i \max(0, -\Delta H_p(B_i, \text{decode})) / \Delta H_s$$

Each word's value is the sum of cost reductions across all receivers who benefit. A definition that helps twelve novices by four ops each has audience efficiency of forty-eight, even though it helps zero experts. The optimal message includes every word whose audience-weighted efficiency exceeds one.

The optimal encoding for a heterogeneous audience has a definite character. It is always longer than the optimal encoding for the most expert receiver (the experts would prefer compressed shorthand). It is always shorter than the optimal encoding for the most novice receiver (full tutorial verbosity wastes channel capacity on explanations that most receivers don't need). The optimal length increases with audience heterogeneity — the wider the spread of dissolution states, the more dissolution infrastructure the message must carry.

This produces a formal explanation for a universal frustration: documentation that satisfies no one. The expert finds it too verbose (they don't need the explanations, and the extra words consume their reading time). The novice finds it too terse (it doesn't provide enough dissolution infrastructure for terms they haven't dissolved). Both are correct — the documentation is suboptimal for both of them. No single encoding optimizes for both simultaneously because their dissolution states differ. The gap between their needs is the dissolution differential between the most and least expert receivers, and no single linear encoding can bridge it optimally.

8. Layered Encoding

The heterogeneous audience problem has no optimal solution in a single linear encoding. But it has an excellent solution in a layered one.

Define a **layered encoding** as a message structure where content is organized in layers of increasing dissolution infrastructure. The base layer contains compressed expert content — minimal, jargon-dense, high compression ratio. Each subsequent layer adds dissolution infrastructure for progressively less-dissolved receivers — definitions, examples, analogies, prerequisite explanations.

The key property of layered encoding: each receiver consumes only the layers they need. The expert reads the base layer and stops — minimal channel cost, zero processing cost. The intermediate reader reads the base layer plus one layer of definitions — moderate channel cost, low processing cost. The novice reads all layers — high channel cost, but reduced processing cost from the dissolution infrastructure in each layer.

Define the **effective cost** of a layered encoding for receiver B_i :

$$\text{Cost}(B_i, \text{layered}) = Hs(\text{layers consumed by } B_i) + Hp(B_i, \text{decode}(\text{consumed layers}))$$

Each receiver's cost includes only the channel bits they actually consume plus their decoding cost for the content they read. The total cost across the audience is:

$$\text{Cost_total}(\text{layered}) = Hp(A, \text{encode_all_layers}) + \sum_i [Hs(\text{layers_consumed}(B_i)) + Hp(B_i, \text{decode})]$$

The sender pays the full encoding cost for all layers. Each receiver pays the channel cost only for the layers they consume. This breaks the single-encoding constraint and allows the optimization to approach the per-receiver optimum for each member of the audience.

Layered encoding has concrete implementations across communication media. In documentation: progressive disclosure, expandable sections, linked glossaries, “Learn more” references. The main page is the base layer. Each expandable section or link is an additional layer of dissolution infrastructure. In technical writing: executive summary (compressed, for dissolved readers), detailed explanation (moderate expansion), appendix with fundamentals (maximum dissolution infrastructure). In API design: simple interface for common cases (base layer), configuration options for advanced cases (second layer), extension points for edge cases (deepest layer). In user interfaces: default view with dissolved conventions (base layer), tooltips (second layer), help documentation (deepest layer). In teaching: lecture overview (base layer), worked examples (second layer), pre-requisite review (deepest layer).

The formal advantage of layered encoding is measurable. For a heterogeneous audience with dissolution profile spanning a wide range, the total audience cost of a layered encoding is lower than the total cost of any single linear encoding. The improvement is proportional to the audience heterogeneity — the more diverse the dissolution states, the greater the benefit of layering.

Define the **layering benefit**:

$$\text{benefit} = \text{Cost}(\text{linear_optimal}) - \text{Cost}(\text{layered_optimal})$$

where both costs are summed across the full audience. The benefit is zero when the audience is homogeneous (all receivers at the same dissolution state — a single encoding suffices). The benefit increases with heterogeneity, bounded above by the difference between full-tutorial cost (every receiver gets maximum infrastructure) and per-receiver optimal cost (each receiver gets exactly what they need).

9. Documentation Quality

Documentation quality is not a matter of opinion. It is a measurable property of the relationship between a document, its content, and its reader population.

Define **documentation quality** for a specific reader population:

$$Q(\text{doc}, \text{readers}) = \text{content_transmitted} / [H_s(\text{doc}) + \sum_i H_p(\text{reader}_i, \text{decode}(\text{doc}))]$$

Quality is the ratio of information successfully communicated to total cost incurred. High quality means the document communicates its content at low combined channel and processing expense. Low quality means readers spend excessive ops decoding, or the document is longer than necessary, or both, or the content doesn’t arrive despite the expense.

This definition captures the informal judgments practitioners make about documentation.

“Too terse” means H_s is low but $\sum H_p(B)$ is high. The document conserves channel bits at the expense of reader processing. It reads quickly for experts but fails novices. The total cost is high because the receiver term dominates.

“Too verbose” means H_s is high without corresponding reduction in $\sum H_p(B)$. Extra words that don’t reduce any reader’s processing cost — repetition that doesn’t clarify, explanations of concepts all readers have dissolved, tangential content that adds channel cost without adding understanding. The total cost is high because the channel term inflated without benefit.

“Poorly organized” means the same total words (same H_s) would achieve lower $\sum H_p(B)$ in a different arrangement. The dissolution infrastructure exists but is in the wrong place — definitions appear after the terms they define, prerequisites are explained after the content that requires them, examples precede the concepts they illustrate. The channel cost is fixed but the processing cost is inflated by poor sequencing.

“Well-written” means the optimization surface has been navigated effectively. Every word either carries content (necessary for H_s) or reduces reader processing (dissolution infrastructure with positive efficiency). No necessary infrastructure is missing (no reader gaps that a word could fill). The encoding sits near the total-cost minimum for the target reader population.

Documentation quality is reader-relative. The same document has different quality for different populations. A terse API reference is high-quality for experts (zero decompression cost, minimal channel waste) and low-quality for novices (high decompression cost, missing infrastructure). A verbose tutorial is high-quality for novices and lower-quality for experts. This is not a defect — it is a mathematical consequence of receiver-dependent processing entropy.

The practical consequence: documentation quality can be measured by measuring reader processing cost. Track how long readers spend per page (proxy for $H_p(B, \text{decode})$). Track how often readers navigate away to look up definitions (direct evidence of decompression cost for specific tokens). Track completion rates (readers who abandon have processing cost exceeding their time budget). Track comprehension outcomes (whether the processing resulted in correct understanding). Each measurement is a component of the total cost, and their sum compared against the content successfully transmitted is the quality score.

10. Teaching Effectiveness

Teaching is communication with a unique property: the sender’s goal is not just to transmit content but to change the receiver’s dissolution state. The teacher aims to move tokens from high decompression cost to low cost — to dissolve concepts in the student’s processing.

Define **teaching effectiveness** as the rate of dissolution in the student as a result of the teaching interaction:

$$E(\text{teacher}, \text{student}, t) = -dH_p(\text{student}, \text{domain}) / dt_{\text{teaching}}$$

where the derivative is taken during the teaching period. Effective teaching reduces the student's processing entropy rapidly. Ineffective teaching leaves it unchanged — the student's decompression costs are the same after the lesson as before.

The three-term cost applies to each teaching interaction. But teaching has a structural feature that distinguishes it from static communication: the receiver's dissolution state changes during the interaction. The decompression cost for a token at the beginning of a lesson may differ from the cost at the end — because the lesson itself may have partially dissolved that token.

This means the optimal encoding for a teaching interaction is time-varying. At the start, the student's dissolution state requires extensive infrastructure — every new concept needs definition, example, and context. As the lesson progresses and the student dissolves initial concepts, the optimal encoding can compress — newly dissolved concepts become building blocks, requiring no further explanation. Later concepts can reference earlier ones at zero decompression cost because the earlier ones were dissolved during the lesson.

Define the **dissolution trajectory of a lesson** as the student's processing entropy profile over the lesson duration:

$$H_{\text{lesson}}(t) = \text{processing entropy at time } t \text{ during the lesson}$$

A well-designed lesson produces a monotonically decreasing trajectory — each new concept builds on previously dissolved ones, and the student's total decompression cost decreases throughout. A poorly designed lesson may produce a non-monotone trajectory — introducing a concept that depends on prerequisites the student hasn't dissolved forces processing entropy upward before the prerequisites can be established, creating a temporary spike that may never resolve if the student's pipeline is overwhelmed.

The concept ordering problem in teaching is now formal. Given a set of concepts to teach, each with prerequisites (other concepts whose dissolution reduces this concept's decompression cost), find the ordering that produces the steepest monotonic decrease in the student's processing entropy profile. This is a topological sort weighted by dissolution efficiency: teach prerequisites first, then concepts that build on the most already-dissolved prerequisites, creating a cascade of dissolution where each new concept's decompression cost is reduced by all previously dissolved concepts.

The formal teaching optimization: given a student's current dissolution state and a target state, find the sequence of encodings (explanations, examples, exercises) that moves the student from current to target at minimum total cost. This is a dynamic programming problem — the optimal encoding at each step depends on the student's dissolution state at that step, which depends on the encodings of all previous steps. The state space is the student's processing entropy profile, the actions are encoding choices, and the cost function is the three-term equation evaluated at each step.

Practical teaching quality follows from this framework. A teacher who accurately estimates the student's dissolution state and adjusts encoding in real time is navigating the

optimization surface dynamically — expanding explanations when the student’s processing entropy is high, compressing when concepts dissolve. The ability to estimate dissolution state from observable signals (questions, expressions, errors, response speed) and adjust encoding accordingly is the core skill formalized.

11. API Design

An API is a message from the designer to the consumer, transmitted through code. The three-term cost equation applies directly, and the quantities are precisely measurable.

Every API call has a processing entropy for each consumer:

$H_p(\text{consumer}, \text{invoke}(\text{call})) = \text{ops to correctly invoke the API call to achieve the consumer's goal}$

An experienced consumer familiar with the API’s conventions invokes a call at near-zero processing entropy. The parameter structure is dissolved — they know what to pass without looking it up. The return type semantics are dissolved — they know what to expect. The error handling is dissolved — they know what can fail and how. A new consumer faces high processing entropy for the same call — they must read documentation (a nested communication with its own three-term cost), understand parameter semantics, handle unfamiliar error types, and interpret return values.

The API’s design determines how processing entropy distributes across the consumer population. Design choices map to the optimization surface:

Consistent naming. When all functions follow a dissolved convention (`get_X`, `set_X`, `create_X`, `delete_X`), the consumer dissolves the convention once and applies it everywhere. Each new function’s decompression cost drops because the naming pattern is already dissolved. Inconsistent naming prevents this dissolution — each function must be individually decoded.

Predictable parameters. When parameters follow a dissolved ordering convention (resource first, options second, callback last), the consumer dissolves the ordering once. Unpredictable ordering forces per-call decoding.

Clear error types. When errors form a dissolved taxonomy (`NotFound`, `PermissionDenied`, `InvalidArgument`, `Timeout`), each error’s decompression cost is low because the taxonomy is dissolved. Opaque error codes force per-error lookup.

Sensible defaults. When default parameter values handle the common case, the consumer can invoke the common case at zero configuration cost — the default path is dissolved. Required parameters for every case force the consumer to understand and specify all options even for common usage.

Each of these conventions reduces receiver processing entropy across the consumer population. Each is a form of dissolution infrastructure — a structural property of the API that provides, at design time, the decompressed associations that experienced consumers

would eventually dissolve through use but that new consumers would otherwise pay for in ops.

Define **API quality** for a consumer population analogously to documentation quality:

$$Q(\text{API, consumers}) = \text{functionality_accessed} / \sum_i \sum_j \text{Hp}(\text{consumer}_i, \text{invoke}(\text{call}_{-j})) \times \text{freq}(\text{call}_{-j}, \text{consumer}_i)$$

Quality is the ratio of functionality successfully accessed to total processing entropy expended, weighted by call frequency. High-quality APIs deliver more functionality per op of consumer processing. Low-quality APIs consume consumer ops on decoding, debugging invocation errors, and reading documentation rather than on the consumer's actual task.

The heterogeneous audience problem applies directly. The API serves experts who want power and brevity and novices who want clarity and guidance. The layered encoding solution applies: a simple surface for common cases (high-level functions with good defaults — the base layer), intermediate control for sophisticated use (configuration parameters, option structs — the second layer), and escape hatches for edge cases (low-level access, extension points, raw mode — the deepest layer). Each layer serves a different dissolution state in the consumer population.

12. Compression Ratio Dynamics

The compression ratio — referents per token for a given processor — changes with experience. Formalizing the dynamics completes the model of how communication efficiency evolves.

Define the **compression maturity curve** for a token:

$$\text{ratio}(\text{token}, \text{processor}, t) = \text{number of referents decompressible from token at time } t$$

The curve starts at first encounter with few referents. A child first learning “fire” decompresses it to one or two referents — the hot dangerous thing, perhaps the bright flickering thing. Through exposure across contexts, new referent associations dissolve: the word in the context of employment, in the context of weapons, in the context of ceramics, in the context of enthusiasm, in the context of urgency. Each new context that dissolves adds a referent to the compression ratio.

The compression maturity curve parallels the dissolution curve but measures a different quantity. The dissolution curve tracks cost reduction for a fixed task — doing the same thing more cheaply over repetitions. The compression maturity curve tracks bandwidth expansion for a fixed symbol — getting more meaning from the same token over exposures. Dissolution is about efficiency. Compression maturity is about capacity.

The two curves connect through a shared mechanism. Dissolving a new referent-to-token association simultaneously: reduces the decompression cost for that referent in the relevant context (dissolution — fewer ops) and adds one referent to the token's compression

ratio for that processor (compression maturity — more meaning per symbol). The underlying event is the same — a new association dissolves — but the two curves measure its different consequences.

Define **codebook alignment** between sender and receiver for a token:

$$\text{alignment}(A, B, \text{token}) = |\mathcal{R}_A(\text{token}) \cap \mathcal{R}_B(\text{token})| / |\mathcal{R}_A(\text{token}) \cup \mathcal{R}_B(\text{token})|$$

where $\mathcal{R}_p(\text{token})$ is the referent set processor p can decompress from the token. This is the Jaccard similarity of their referent sets. Perfect alignment (1.0) means the token decompresses identically for both — every referent the sender intends, the receiver can access, and vice versa. Zero alignment means the token means entirely different things to each.

Codebook alignment aggregated across all tokens in a message predicts communication efficiency:

$$\text{alignment}(A, B, \text{message}) = \text{mean alignment across all tokens in message}$$

High aggregate alignment means the sender and receiver share compressed meaning — the sender's intended referents are the ones the receiver activates. Low alignment means systematic misinterpretation — the sender compresses one meaning, the receiver decompresses a different one, and the channel transmitted the token perfectly while the communication failed entirely.

Low alignment manifests differently from high decompression cost and is a distinct failure mode. High decompression cost means the receiver works hard but eventually arrives at the sender's intended meaning. Low alignment means the receiver arrives at a different meaning without working hard — the decompression fires automatically but to the wrong referent. The receiver doesn't know they've misunderstood because their decompression produced a coherent (but incorrect) result at zero or low cost. This is the most dangerous communication failure: invisible, confident, and wrong.

The dynamics of alignment have a temporal structure. When two people begin working together, their codebook alignment for domain-specific tokens is determined by their respective training and experience — potentially high if they share a background, potentially low if they come from different traditions. Over time, shared experience aligns their codebooks — they dissolve the same referents for the same tokens through shared context. Team formation is, among other things, the progressive alignment of compression codebooks through shared dissolution experiences.

13. Civilization as Accumulated Dissolution

The compression and dissolution dynamics described in this paper operate not just between individuals but across populations and across time. The consequences extend to the structure of accumulated human knowledge.

Writing dissolved the speaker-presence requirement. Before writing, every communication required sender and receiver to be co-located in time and space. The sender's en-

coding cost included being physically present. Writing moved the encoding to a durable medium — the sender pays encoding cost once, and the message transmits across time and space at channel cost only. This is dissolution infrastructure at civilizational scale: one encoding, arbitrarily many decodings.

Printing dissolved the scribe requirement. Before printing, each copy of a written message required a human copyist — encoding cost paid per copy. The printing press dissolved per-copy encoding to near-zero. The sender (author) pays encoding cost once. The press pays mechanical reproduction cost per copy. Readers pay decoding cost individually. The channel cost per reader dropped by orders of magnitude.

Standard notation dissolved per-problem derivation. Mathematical notation — the equals sign, algebraic variables, the integral symbol, the summation symbol — is a compression codebook dissolved across the mathematical community. Each symbol compresses a complex concept into a glyph. The decompression cost for trained mathematicians is zero. Before standard notation, each mathematical argument required natural-language explanation of the operations — high channel cost and high receiver processing cost. Standard notation reduced both: lower channel cost (fewer symbols needed) and lower processing cost (dissolved decompression for trained receivers).

Each advance is the same pattern: a compression token or dissolution infrastructure created once, dissolved across a population, reducing per-communication total cost. The alphabet reduced the infinite space of logograms to composable letters. Positional notation with zero dissolved per-scale arithmetic procedures. Standard time zones dissolved active cross-location time conversion. Standard shipping containers dissolved per-cargo handling procedures.

Civilization, from this framework's perspective, is the accumulated stack of dissolution infrastructure and shared compression codebooks. Each layer was built when some processor invested encoding ops to create a token or structure. That token or structure then dissolved across a population, reducing the population's processing entropy for the relevant tasks. The freed pipeline capacity was then available for the next unsolved problem — which, when solved, created the next layer of dissolved infrastructure.

The rate of civilizational progress is bounded by the rate at which dissolution infrastructure can be created and dissolved across populations. The printing press accelerated this rate by reducing per-copy channel cost. The internet accelerated it again by reducing distribution cost to near-zero. In each case, the acceleration came from reducing the channel term H_s , which freed more of the total cost budget for the processing terms — creation (sender encoding) and learning (receiver dissolution).

14. Scope and Open Problems

This paper establishes the three-term communication cost equation as an optimization framework, defines compression tokens and decompression cost, derives the sender-receiver dissolution asymmetry, formalizes redundancy as dissolution infrastructure with measurable efficiency, poses the heterogeneous audience problem and solves it through

layered encoding, gives formal definitions of documentation quality, teaching effectiveness, and API quality as processing entropy minimization, and models compression ratio dynamics and codebook alignment.

The following remain open.

Dissolution state estimation. The optimal encoding requires the sender to know the receiver's dissolution state. In practice this is estimated from context, role, questions, and observable behavior. Formalizing how accurately senders estimate receiver dissolution state, how estimation error propagates to total cost, and how estimation accuracy can be improved would close the gap between theoretical and achievable optimality.

Dynamic teaching optimization. The teaching optimization is a dynamic programming problem over a changing receiver dissolution state. Characterizing the optimal solution structure — whether optimal teaching sequences have domain-independent properties, whether curriculum design admits general principles beyond topological sorting of prerequisites — is open.

Multi-channel optimization. Real communication often uses multiple channels simultaneously — text, diagrams, speech, gesture, code examples. Each channel has its own Hs and its own effect on receiver processing entropy. A diagram may dissolve a concept that text cannot efficiently convey. The joint optimization across channels — choosing which content to encode on which channel to minimize total cost — is a richer problem than single-channel optimization.

Compression ratio ceiling. The compression maturity curve approaches a ceiling. What determines this ceiling — total referents in the domain, structural connectivity of the referent space, processor memory architecture — is unknown. Formalizing the ceiling would complete the compression maturity model.

Network dissolution dynamics. When many processors dissolve the same codebook, communication efficiency increases network-wide. How shared vocabulary spreads through communities, how jargon adoption reaches tipping points, and how standards emerge are dissolution dynamics at the population level. These dynamics connect the individual processing framework to the sociology of communication.

Total cost measurement protocols. Measuring each term in practice requires operational procedures. Sender encoding cost might be measurable from composition time. Channel cost from message length. Receiver processing cost from comprehension time, comprehension accuracy, or clarification query frequency. Standardized protocols would make the framework empirically testable and comparable across studies.

Appendix: Supporting Tables

HOWL-MATH-19-2026

Table A: Formal Definitions

Symbol	Name	Definition	Unit	Domain
$H_p(A, \text{encode})$	Sender encoding cost	Ops sender A executes to transform internal state into transmissible symbols	ops	Sender-specific
$H_s(\text{channel})$	Channel cost	Bits required for reliable transmission through channel	bits	Channel-specific (Shannon's domain)
$H_p(B, \text{decode})$	Receiver decoding cost	Ops receiver B executes to transform received symbols into actionable understanding	ops	Receiver-specific
$\text{Cost}(A \rightarrow B)$	Total communication cost	$H_p(A, \text{encode}) + H_s(\text{channel}) + H_p(B, \text{decode})$	ops + bits + ops	Sender-receiver pair specific
C	Compression function	Maps referent space to single transmissible token; many-to-one	—	Token-specific
C^{-1}	Decompression function	Maps (token, context) to referent; context-dependent	—	Token-context-processor specific
$\text{ratio}(\text{token}, p)$	Compression ratio	Number of referents processor p can decompress from token across all contexts	count	Processor-token pair
$\text{decomp}(\text{token}, p, \text{ctx})$	Decompression cost	$H_p(p, \text{decode}(\text{token}, \text{ctx}))$; ops to decode token in context	ops	Processor-token-context triple
$\Delta(A, B, \text{tokens})$	Dissolution differential	$\sum_t [H_p(B, \text{decode}(t)) - H_p(A, \text{decode}(t))]$; total cost gap between sender and receiver	ops	Sender-receiver-token set triple

Symbol	Name	Definition	Unit	Domain
redundancy(msg)	Explanatory redundancy	$H_s(\text{message}) - H_s(\text{minimal encoding});$ channel bits beyond information-theoretic minimum	bits	Message-specific
$\eta(\text{word}, B)$	Dissolution efficiency	$-\Delta H_p(B, \text{decode}) / \Delta H_s;$ receiver cost reduction per unit channel cost	ops/bit	Word-receiver pair
$\eta \text{ audi-ence}(\text{word})$	Audience-weighted dissolution efficiency	$\sum_i \max(0, -\Delta H_p(B_i)) / \Delta H_s;$ total receiver cost reduction per unit channel cost	ops/bit	Word-audience pair
encoding*	Optimal encoding	$\text{argmin}_e \{H_p(A, \text{encode}(e)) + H_s(e) + H_p(B, \text{decode}(e))\};$ total-cost-minimizing encoding	—	Sender-receiver pair
alignment(A,B,tokens)	Codebook alignment	Jaccard similarity of referent sets: $ R \cap B / R \cup B $	dimensionless [0,1]	Sender-receiver-token triple
Q(doc, readers)	Documentation quality	content transmitted / $[H_s(\text{doc}) + \sum_i H_p(\text{reader}_i, \text{decode}(\text{doc}))]$	content/cost	Document-reader population pair
E(teacher, student, t)	Teaching effectiveness	$-dH_p(\text{student}, \text{domain}) / dt$ teaching; dissolution rate during teaching	ops/time	Teacher-student-time triple
Q(API, consumers)	API quality	functionality accessed / $\sum_i \sum_j H_p(\text{consumer}_i, \text{invoke}(\text{call}_j)) \times \text{freq}(\text{call}_j)$	functionality/ops	API-consumer population pair
ratio(token, p, t)	Compression maturity	Number of referents decompressible from token at time t; grows with experience	count	Processor-token-time triple

Symbol	Name	Definition	Unit	Domain
benefit(layered)	Layering benefit	Cost(linear optimal) – Cost(layered optimal); total cost saved by layered encoding	ops + bits	Audience-specific

Table B: Three-Term Cost Scenarios

Scenario	Hp(A, encode)	Hs(channel)	Hp(B, decode)	Dominant Term	Shannon Sufficient?	Total Cost Character
Expert → Expert (shared domain)	~0 (dis-solved encoding)	Fixed (message length)	~0 (dis-solved decoding)	Channel	Yes — both endpoints dissolved	Minimum possible; channel-limited
Expert → Novice	~0 (dis-solved encoding)	Fixed	High (5–10 ops per undis-solved token)	Receiver decoding	No — receiver term dominates by 10–100 ×	Novice overwhelmed despite perfect channel
Novice → Expert	High (effortful encoding)	Fixed	~0 (expert decodes anything)	Sender encoding	No — sender term dominates	Expert compensates; communication succeeds slowly
Novice → Novice	High	Fixed	High	Both endpoints	No — both terms high	Slowest possible; both sides struggle
Expert → Mixed audience (N receivers)	~0	Fixed	$\sum_i \text{Hp}(B_i)$ — varies widely	Sum of receiver terms	No — scales with audience heterogeneity	Documentation problem: no single encoding optimal

Scenario	Hp(A, encode)	Hs(channel)	Hp(B, decode)	Dominant Term	Shannon Sufficient?	Total Cost Character
Machine → Ma- chine (APIs)	~0 (com- piled encod- ing)	Fixed (proto- col)	~0 (com- piled decod- ing)	Channel	Yes — both endpoints pre- compiled	Shannon's original model; closest real-world match
Teacher → Stu- dent (over les- son)	Moderate (adapt- ing encod- ing)	Increases over lesson	Decreasing (student dissolv- ing)	Shifts from receiver to channel over time	Initially no; approaches yes as student dissolves	Dynamic op- timization; encoding should adapt
Cross- cultural	Moderate (unfamil- iar encod- ing conven- tions)	Fixed	High (cultural decom- pression) + align- ment failures	Receiver decoding + align- ment	No — both processing and alignment failures	Most failure modes simul- taneously present

Table C: Dissolution Differential by Communication Type

Communication Type	Typical Sender	Typical Receiver	Token Cate- gories	Typical Differential (ops)	High- Differential Tokens	Intervention
Senior architect → junior developer	15+ years domain expert	0–2 years ex- perience	Architecture patterns, infras- tructure terms, team jargon	50–400 per document page	System- specific names, architecture pattern names, implicit team conventions	Glossary; archi- tec- ture decisi- on records; on- board- ing docu- menta- tion

Communication Type	Typical Sender	Typical Receiver	Token Categories	Typical Differential (ops)	High-Differential Tokens	Intervention
Specialist physician → patient	Domain expert	Zero domain knowledge	Medical terminology, procedure names, drug names, anatomical terms	200–500 per consultation	All medical jargon; anatomical references; probabilistic language	Plain language; analogies; diagrams; teach-back verification
Professor → undergraduate	Research expert	Introductory knowledge	Technical vocabulary, theoretical frameworks, methodological terms	100–300 per lecture	Newly introduced terms; terms with everyday meanings used technically	Definitions at point of first use; examples; prerequisite review
API documentation → new consumer	API designer	First-time user	API-specific names, parameter conventions, error taxonomies, architectural assumptions	50–150 per API endpoint	Endpoint naming conventions; authentication patterns; error code meanings	Quick-start guide; code examples; interactive playground

Communication Type	Typical Sender	Typical Receiver	Token Categories	Typical Differential (ops)	High-Differential Tokens	Intervention
Crossteam Slack message	Domain A expert	Domain B expert	Domain A jargon, project-specific abbreviations, assumed context	30–80 per message thread	Team-specific acronyms; project code names; implicit references to past decisions	Expand acronyms; provide context links; avoid assumed shared knowledge
Regulatory text → general public	Legal expert	Non-specialist	Legal terms of art, procedural references, jurisdictional conventions	300–600 per page	Latin phrases; defined terms used without definition; cross-references to other regulations	Plain language summary; layered document with simplified overview
Research paper → adjacent field	Specialist in field X	Specialist in field Y	Field X jargon, methodological assumptions, canonical references	80–200 per paper	Field-specific method names; assumed baseline knowledge; in-group references	Extended introduction; cross-field analogies; explicit methodology

Communicative Type	Typical Sender	Typical Receiver	Token Categories	Typical Differential (ops)	High-Differential Tokens	Intervention
Parent → child	Adult fluency	Developing vocabulary	Abstract concepts, temporal reasoning, causal language, social conventions	Varies enormously by age	Any word the child hasn't encountered; abstract temporal/causal concepts	Simplified vocabulary; concrete examples; repetition; embodied demonstration

Table D: Dissolution Efficiency by Infrastructure Type

Infrastructure Type	Mechanism	Typical η (ops saved per bit added)	Best For	Worst For	Channel Cost (bits)	Processing Reduction (ops)
Inline definition	Defines term at point of first use	3–8	Single unfamiliar term; reader proceeds without interruption	Terms reader already knows (zero efficiency)	5–15 words	5–10 ops per reader who needs it

Infrastructure Type	Mechanism	Typical η (ops saved per bit added)	Best For	Worst For	Channel Cost (bits)	Processing Reduction (ops)
Example	Concrete instance of abstract concept	5–15	Abstract concepts; pattern illustration; connecting to known experience	Concepts that don't generalize from examples; readers who already understand abstraction	20–100 words	10–30 ops if example connects to reader's dissolved experience
Analogy	Maps unfamiliar concept to familiar one	8–20	Structural concepts; cross-domain communication; building intuition	Misleading when analogy breaks down; readers familiar with the target concept	10–30 words	15–25 ops if source domain is dissolved for reader

Infrastructure Type	Mechanism	Typical η (ops saved per bit added)	Best For	Worst For	Channel Cost (bits)	Processing Reduction (ops)
Diagram	Visual representation of structure or process	10–30	Spatial relationships; process flows; multi-components	Sequential logic; abstract arguments; readers with visual processing difficulties	Varies (image bytes)	15–40 ops for structural/spatial content
Code example	Executable demonstration	5–25	API usage; algorithm illustration; concrete behavior specification	Readers who can't read the programming language; overly simplified examples	10–50 lines	10–30 ops if reader's language is dissolved

Infrastructure Type	Mechanism	Typical η (ops saved per bit added)	Best For	Worst For	Channel Cost (bits)	Processing Reduction (ops)
Prerequisite review	Brief coverage of assumed background	2–5	Heterogeneous audiences; topics with strict pre-requisite site chains	Audiences where all readers have pre-requisites (wastes channel)	50–200 words	20–50 ops per reader missing pre-requisites
Glossary	Collected definitions accessible by reference	1–3 per lookup	Reference material; ongoing use; heterogeneous audiences who need different terms	First-time linear reading (interrupts flow); dis-solved readers (never used)	5–15 words per entry	3–8 ops per lookup, but requires reader to navigate

Infrastructure Type	Typical η (ops saved per bit added)		Best For	Worst For	Channel Cost (bits)	Processing Reduction (ops)
Summary / TL;DR	Compressed overview of full content	3–6	Time-constrained readers; readers assessing relevance; providing advance organizer	Readers who need full detail; topics that don't compress well	20–50 words	10–20 ops by providing structural scaffold for subsequent reading
Worked solution	Step-by-step demonstration of problem-solving	8–15	Procedural knowledge; mathematically/algorithmically domains; skill building	Declarative knowledge; readers who think better by attempting first	30–100+ words/steps	15–40 ops by making implicit reasoning explicit

Infrastructure Type	Typical η (ops saved per bit added) Mechanism	Best For	Worst For	Channel Cost (bits)	Processing Reduction (ops)
FAQ	Common questions with answers	Known confusion points; recurring support requests; heterogeneous audiences	Novel questions not in FAQ; rapidly changing content	20–50 words per entry	5–15 ops per applicable question

Table E: Heterogeneous Audience Optimization

Audience Composition	Dissolution Profile	Optimal Strategy	Message Length vs Expert-Optimal	Key Tradeoff
All experts (homogeneous)	All near zero for domain tokens	Compressed shorthand; minimal encoding	$1 \times$ (expert-optimal = overall optimal)	None; single encoding suffices
All novices (homogeneous)	All high for domain tokens	Full tutorial; maximum dissolution infrastructure	$5\text{--}15 \times$ expert-optimal	None; single encoding suffices, just long
Experts + novices (bimodal)	Two clusters: near-zero and high	Layered: compressed base + expandable infrastructure	$2\text{--}4 \times$ expert-optimal (base + one layer)	Expert reading time vs novice comprehension

Audience Composition	Dissolution Profile	Optimal Strategy	Message Length vs Expert-Optimal	Key Tradeoff
Continuous spread (uniform)	Even distribution from zero to high	Layered with multiple tiers; progressive disclosure	$3-8 \times$ expert-optimal (multiple layers)	Number of layers vs navigation complexity
Mostly experts, few novices	Cluster near zero, sparse high outliers	Compressed with linked glossary/appendix	$1.2-2 \times$ expert-optimal	Minimal main-text disruption vs novice support
Mostly novices, few experts	Cluster at high, sparse near-zero	Expanded with expert fast-paths (skip-ahead markers)	$4-10 \times$ expert-optimal with skip navigation	Expert scanning efficiency vs novice support
Multiple distinct domains (cross-functional)	Clustered by domain; each dissolved in own area, not others	Domain-specific sections with shared overview	$2-5 \times$ single-domain optimal	Shared concepts vs domain-specific vocabulary
Unknown audience	Distribution unknown	Layered with broad coverage; progressive disclosure at every level	$3-6 \times$ expert-optimal (hedge across possibilities)	Over-investment in infrastructure vs risk of underserving any subgroup

Table F: Documentation Quality Metrics

Metric	What It Measures	Measurement Method	Relationship to Three-Term Cost	Quality Signal
Time on page	Total reading time including de-compression	Web analytics; reading time tracking	Proxy for H_s (reading time for channel) + $H_p(B, \text{decode})$ (processing time for decompression)	High time may be good (deep engagement) or bad (confusion); disambiguate with other metrics
Definition lookup rate	Frequency of navigating to glossary or external definitions	Click tracking; navigation analysis	Direct measure of per-token decompression cost; each lookup = failed dissolution for that token	High rate = high dissolution differential; terms need inline definitions
Completion rate	Fraction of readers who reach end of document	Scroll depth tracking; page analytics	Proxy for total cost vs time budget; incomplete = total cost exceeded budget	Low completion = message too expensive for audience; shorten or add infrastructure
Comprehension score	Accuracy of understanding after reading	Quiz; task performance; teach-back	Inverse of residual processing entropy after decoding; high score = low remaining $H_p(B)$	Low score = decoding failed despite channel delivery; information transmitted but not processed

Metric	What It Measures	Measurement Method	Relationship to Three-Term Cost	Quality Signal
Return visits	How often readers revisit the same document	Visit frequency tracking	High returns = incomplete dissolution per visit; multiple visits needed to dissolve content	Moderate returns normal for complex content; excessive returns = content not structured for dissolution
Time to first action	Duration from reading to applying information	Workflow tracking; task initiation timestamps	Total cost from reading through actionability; includes decoding + reduction to actionable One	Short time = content reached actionability quickly; long time = additional processing needed after reading
Search-after-reading rate	How often readers search for related content after reading	Search log analysis	Indicates content didn't provide sufficient dissolution infrastructure; reader seeking supplementary decompression	High rate for specific terms = targeted infrastructure gaps
Reader satisfaction (qualitative)	Subjective quality assessment	Survey; ratings	Correlates with but is not identical to cost minimization; satisfaction reflects perceived cost-to-value ratio	Useful as overall indicator; decompose with other metrics for actionable insight

Table G: Teaching Interaction Dynamics

Phase	Student Dissolution State	Optimal Encod- ing	Dissolution Infrastructure Needed	Encoding Density	Processing Entropy Trajectory
Opening / mo- ti- va- tion	Baseline; topic undissolved; related concepts may be partially dissolved	Connect to dissolved prior knowl- edge; use familiar vocabu- lary exclu- sively; motivate why topic matters	Maximum — every new concept requires full context	Low; mostly dissolved vocabu- lary with careful framing	Stable or slight decrease (connecting to existing dissolutions)
First con- cept in- tro- duc- tion	Baseline for new concept; prerequisites being activated	Define explic- itly; single new concept per unit; concrete example immedi- ately following defini- tion	High — definition + example + connection to prior knowledge per concept	Low to moder- ate; one new token per para- graph	Decreasing; first concept dissolving

Phase	Student Dissolution State	Optimal Encod- ing	Dissolution Infrastructure Needed	Encoding Density	Processing Entropy Trajectory
Building on first concept	First concept partially dissolved; de-compression cost decreasing	Begin using first concept as token (test whether it's dissolved enough to build on); introduce second concept using first as scaffold	Moderate — first concept carries some load; second concept needs full infrastructure	Moderate; increasing compression ratio	Decreasing; accelerating as building blocks accumulate
Mid-lesson acceleration	Several concepts dissolved; building blocks available	Increase encoding density; use dissolved concepts freely; introduce concepts faster; examples can be shorter	Decreasing — most infrastructure is previously dissolved concepts	Moderate to high; approaching expert encoding for covered concepts	Decreasing steeply; dissolution cascade (each new concept cheaper because of dissolved prerequisites)

Phase	Student Dissolution State	Optimal Encod- ing	Dissolution Infrastructure Needed	Encoding Density	Processing Entropy Trajectory
Practice / application	Concepts introduced; dissolution in progress but incomplete	Shift from encoding to exercises; student generates rather than receives; errors reveal gaps	Minimal from teacher; student provides own dissolution through practice	Low teacher encoding; student self-encoding through problem-solving	Decreasing through practice; approaching zero for practiced elements
Consolidation	Lesson concepts at low decomposition cost	Restate in compressed form; connect to broader context; preview how concepts enable future learning	Minimal; summary uses tokens that are now dissolved	High; compressed summary using lesson's dissolved vocabulary	Stable near zero for lesson content; slight increase as connections to broader context reveal unseen territory

Table H: API Design Patterns as Processing Entropy Optimization

Design Pattern	Sender Cost (designer)		Channel Cost (API surface)	Receiver Cost (consumer)	Total Cost Optimization
Consistent naming (get X, create X, delete X)	Convention: dis-solves once; applies to all end-points	Low: convention defined once	Moderate: longer names than abbreviations	Low: naming pattern dissolved; new endpoints de-compressible from pattern	Minimizes receiver cost per endpoint at small channel cost increase
Predictable parameter ordering (re-source, options, call-back)	Convention: dis-solves once; applies to all calls	Low: ordering defined once	Neutral: same parameters, different order	Low: ordering pattern dissolved; invocation is structural	Minimizes receiver cost per call at zero channel cost
Descriptive error types (NotFound, InvalidArgument, Timeout)	Error taxonomy dis-solves once; each error immediately meaningful	Moderate: taxonomy design requires thought	Moderate: more types than opaque codes	Low: error handling pattern dissolved; failures de-compressible	Reduces receiver debugging ops at moderate design cost

Design Pattern	Mechanism	Sender Cost (designer)	Channel Cost (API surface)	Receiver Cost (consumer)	Total Cost Optimization
Sensible defaults	Common case requires zero configuration; advanced cases progressive	High: identifying common case requires research	Reduced: fewer required parameters for common path	Very low for common case; moderate for advanced	Minimizes common-case receiver cost at sender design cost
Progressive disclosure (simple → intermediate → advanced)	Layered API surface; consume only needed complexity	High: multiple API layers require more design	Higher: larger total surface	Optimal per consumer: each uses their layer	Heterogeneous audience optimization via layering
Code examples in documentation	Executable disolution infrastructure	Moderate: writing correct examples	Moderate: examples add doc length	Low to very low: copy-paste-modify path dissolved	Reduces receiver first-use cost dramatically

Design Pattern	Mechanism	Sender Cost (designer)	Channel Cost (API surface)	Receiver Cost (consumer)	Total Cost Optimization
Interactive play-ground / REPL	Zero-cost experimentation; receiver builds dissolution through self-directed practice	High: building interactive environment	High: hosting + maintenance	Very low: self-directed dissolution through practice	Highest total receiver cost reduction; highest sender investment
Backward compatibility	Existing dissolved invocations remain valid; no cascade from version change	Very high: maintaining compatibility constrains design	Higher: old and new surfaces maintained	Zero for existing consumers; prevents dissolution cascade from version change	Protects existing consumer dissolution investment at sender cost

Table I: Compression Ratio Examples

Token	Processor	Compression Ratio	Sample Referent Set	Decompression Cost	Context Sensitivity
“fire”	3-year-old child	2–3	Hot/dangerous thing; fire truck; “stop that”	Zero for known referents; undefined for others	Low; limited contexts experienced
“fire”	General adult	12–18	Combustion; terminate employment; shoot weapon; kiln process; enthusiasm; urgency; campfire; fireplace; fire alarm; fire escape; fire department; fire sale	Zero for common referents; 1–2 ops for rare	Moderate; most contexts automatically resolved
“fire”	Firefighter	40–60	All adult referents + structural fire behavior; ventilation; flashover; backdraft; fire load; Class A/B/C/D/K; suppression tactics; incident command; PPE; SCBA; RIT; accountability; exposure protection	Zero for professional referents	High; professional context triggers extended set

Token	Processor	Compression Ratio	Sample Referent Set	Decompression Cost	Context Sensitivity
“fire”	Arson investigator	80–120	All firefighter referents + accelerant patterns; pour patterns; V-patterns; char depth; electrical origins; ignition devices; evidence preservation; chain of custody; expert testimony; insurance fraud indicators	Zero for professional referents	Very high; investigative context triggers specialized set
“buffer”	Non-technical adult	1–2	Something that absorbs impact; waiting area	Zero	Low
“buffer”	Junior developer	4–6	Memory region; queue; I/O staging; string builder; video streaming preload	Zero for common; 2–3 ops for systems concepts	Moderate

Token	Processor	Compression Ratio	Sample Referent Set	Decompression Cost	Context Sensitivity
“buffer”	Systems pro-gram-mer	15–25	All developer referents + ring buffer; circular buffer; zero-copy; buffer overflow; buffer pool; double buffering; write-behind; page cache buffer; network buffer; DMA buffer; pipe buffer; TTY buffer; framebuffer	Zero for all	High; substrate context selects specific type
“normal”	General public	3–5	Typical/average; perpendicular (math, rarely used); not abnormal (medical, rarely)	Zero for “typical”; 2–3 ops for technical	Low
“normal”	Mathematician	1–2	Perpendicular; normal distribution; normal subgroup; normal vector; surface normal; normal form; normalize; unit normal; normal bundle	Zero for all; context resolves instantly	Very high; each mathematical context selects precisely

Token	Processor	Compression Ratio	Sample Referent Set	Decompression Cost	Context Sensitivity
"normal	Physician	6–10	Within reference range; no pathology detected; normal variant; normal saline; normal flora; normalize (lab values)	Zero for all clinical referents	High; clinical context selects precisely

Table J: Codebook Alignment Failure Modes

Alignment Level	Character	Communication Effect	Example	Detection	Resolution
1.0 (perfect)	Sender and receiver decompress identically for all message tokens	Communication succeeds; minimal processing cost at both endpoints	Two cardiologists discussing an EKG; two Python developers discussing list comprehensions	N/A	N/A

Alignment Level	Character	Communication Effect	Example	Detection	Resolution
0.8–0.99 (high)	Most tokens aligned; few mis-aligned tokens identifiable	Communication mostly succeeds; occasional misunderstanding on specific terms	Developers from same company, different teams; physicians from same specialty, different hospitals	Misunderstanding surface during implementation or action; specific terms identified as sources	Clarify specific mis-aligned tokens; build shared glossary
0.5–0.8 (moderate)	Many tokens partially aligned; some referents shared, others not	Communication partially succeeds; significant chunks require clarification	Cross-functional meeting (engineer + designer + PM); adjacent medical specialties	Frequent clarification requests; visible confusion on specific topics; different conclusions from same discussion	Explicit definitions at interaction boundary; shared vocabulary building; layered encoding

Alignment Level	Character	Communication Effect	Example	Detection	Resolution
0.2–0.5 (low)	Few tokens aligned; most decompress to different referents	Communication largely fails; both parties think they understand but reach different conclusions	Cross-cultural business negotiation; physician explaining to patient using medical jargon; academic writing for general audience	Actions taken don't match intent; systematic misinterpretation discovered downstream	Complete vocabulary reset; use only tokens known to be aligned; verify comprehension at each step

Alignment Level	Character	Communication Effect	Example	Detection	Resolution
0.0–0.2 (minimal)	Almost no tokens aligned; same words mean different things to each party	Communication fails entirely or produces dangerous misunderstanding; receiver confident in wrong interpretation	Legal language read by non-lawyer; technical specification read by marketing team without technical background; regulatory text in foreign legal tradition	Catastrophic down-stream failure; actions taken are exactly wrong; no awareness of misunderstanding during communication	Translator/intermediary with dual dissolution; complete re-encoding in receiver's vocabulary; abandon token-based communication for demonstration

Table K: Layered Encoding Implementations

	Base Layer Medium(Expert)	Second Layer (Intermediate)	Third Layer (Novice)	Navigation Mechanism	Implementation Cost
Technical documentation	Compressed reference; API signatures; parameter tables; terse descriptions	Conceptual explanations; usage examples; common patterns; best practices	Tutorials; prerequisite explanations; glossary; step-by-step guides; analogies	Table of contents with difficulty markers; “Getting started” vs “Reference” navigation	Moderate; 2–3 × single-document effort
Web documentation	Collapsed/mini-default view; dense content	Expandable sections; tooltip definitions; inline examples	Linked tutorials; prerequisite pages; video walk-throughs; interactive playgrounds	Expand/collapse progressive disclosure; depth-based navigation	Moderate to high; requires frontend engineering
Academic paper	Abstract + results summary; equations; compressed methodology	Extended introduction; worked examples; methodology details	Background section; notation guide; appendix with derivations; supplementary materials	Paper structure itself is layered; supplementary materials for deeper layers	Low additional cost; follows established format
Presentation / lecture	Slide title + key insight (one sentence per slide)	Verbal explanation accompanying slide; moderate detail	Q&A; supplementary handout; recorded lecture for review; office hours	Time-based progression; Q&A as demand-driven infrastructure	Moderate; preparation time for materials at each layer
API design	Simple function calls with good defaults; minimal parameters	Configuration objects; option parameters; builder patterns	Tutorials; cookbooks; interactive REPL; migration guides	Function signature simplicity → configuration depth → documentation depth	High; multiple interface layers require careful design

	Base Layer Medium(Expert)	Second Layer (Intermediate)	Third Layer (Novice)	Navigation Mechanism	Implementation Cost
User in- ter- face	Primary action buttons; dissolved conventions (standard icons)	Tooltips; contextual help; onboarding overlays	Help docu- mentation; tutorial mode; guided walk- throughs; support chat	Hover → click → navigate depth progression	High; UI engineering at each layer
Email / mes- sage	Subject line (one- sentence summary)	First paragraph (key content and action items)	Full body (context, reasoning, supporting detail, background)	Reading by scanning: subject → first paragraph → full body as needed	Low; writer discipline only
Codebase	Function signatures and type system (self- documenting code)	Inline comments at decision points; doc-comments on public interfaces	Architecture documents; README; contribution guide; code walk- throughs	Code → comments → documen- tation hierarchy	Moderate; ongoing mainte- nance across layers

Table L: Cross-Domain Communication Applications

Domain	Sender	Receiver	Primary Optimization Challenge	Dominant Cost Term	Key Intervention
Technical writ- ing	Domain ex- pert au- thor	Heterogeneous reader- ship from novice to expert	Heterogeneous audience; no single optimal encoding	$\sum H_p(B_i, \text{decode})$ — sum of reader decoding costs	Layered encoding; progressive disclosure

Domain	Sender	Receiver	Primary Optimization Challenge	Dominant Cost Term	Key Intervention
Medical communication	Physician	Patient (zero domain knowledge)	Maximum dissolution differential; high consequence of misunderstanding	Hp(patient, decode) — enormous per token	Plain language; analogies to dissolved experience; teach-back verification; diagrams
Legal communication	Lawyer	Client or general public	Legal tokens have precise meanings that differ from everyday usage; alignment near zero	Hp(public, decode) + alignment failures — both processing and misinterpretation	Plain language summaries; glossary of terms used technically; explicit flagging of words used differently than common meaning
Cross-cultural communication	Culture A member	Culture B member	Different dissolved conventions; gestures, metaphors, social protocols have different referents	Alignment failures dominate — confident wrong decomposition	Cultural intermediary; explicit context for conventions; avoid compressed idiom; verify understanding
Classroom teaching	Teacher	Students at varied preparation levels	Time-varying receiver dissolution state; heterogeneous starting points	Hp(student, decode) decreasing over lesson; sum across students with different rates	Dynamic encoding adjustment; formative assessment; differentiated instruction

Domain	Sender	Receiver	Primary Optimization Challenge	Dominant Cost Term	Key Intervention
Presentation to executives	Technical expert	Non-technical decision-maker	Executives have dissolved business vocabulary but not technical vocabulary; time budget extremely tight	Hp(exec, decode) for technical content; time budget constraint	Translate to dissolved business vocabulary; lead with implication not mechanism; single-layer encoding at executive's dissolution level
Open-source README	Project creator	Potential contributors with unknown backgrounds	Unknown audience dissolution profile; first impression determines engagement	Hp(contributor, decode) for first 30 seconds of reading	Immediate concrete example; problem statement in common vocabulary; quick-start before architecture
Scientific peer review	Researcher in field X	Researcher in adjacent field Y	Shared general methodology; different specialized vocabulary; partial codebook overlap	Per-token alignment varying by token: high for shared methodology, low for specialization	Extended methods; define field-specific terms; explicit connections to shared foundations
Emergency communication	Incident commander	Mixed responders (fire, police, medical)	Extreme time pressure; different dissolved vocabularies across agencies; high consequence	Time budget constraint + cross-agency alignment gaps	Standardized plain language (ICS); no jargon; explicit confirmation; structured formats

Table M: Civilization Dissolution Infrastructure Stack

Era	Innovation	What It Dissolved	Dissolution Type	Channel Cost Change	Processing Cost Change	Population Affected
~3000 BCE	Writing	Speaker-presence requirement for communication	Encoding → durable medium	One-time encoding; persistent channel	Receiver pays decode per read; sender pays encode once	Literate elites → gradually expanding
~1500 BCE	Alphabet	Per-concept logograms; thousands of unique symbols	Infinite symbol set → ~26 composable letters	Reduced per-symbol channel cost; increased symbols per message	Reduced learning cost (dissolve 26 letters vs thousands of logograms)	Literate populations
~500 CE	Positional notation with zero	Per-scale arithmetic procedures; abacus dependency	Manual computation → structural notation	Compact number representation	Eliminated per-operation lookup; arithmetic dissolved into notation manipulation	Mathematical practitioners
~1450	Printing press	Per-copy scribe encoding cost	Manual copying → mechanical reproduction	Dramatically reduced per-copy channel cost	Unchanged per-reader; but more readers served	Expanding literate population
~1880	Standard time zones	Active cross-location time conversion	Per-communication time calculation → structural lookup	Added zone identifier to communications	Eliminated 3–5 ops of time conversion per cross-location communication	All cross-region communicators

Era	Innovation	What It Dissolved	Dissolution Type	Channel Cost Change	Processing Cost Change	Population Affected
~1956	Shipping containers	Per-cargo handling procedures	Custom loading → standardized interface	Standardized container dimensions (fixed channel format)	Eliminated per-item handling decisions; dissolved load-ing/unloading to mechanical operation	Global trade
~1970	Internet protocols (TCP/IP)	Per-network communication procedures	Network-specific encoding → universal protocol	Standardized packet format	Eliminated per-network encoding/decoding; dissolved network boundary crossing	All networked computing
~1990	World Wide Web	Per-document distribution procedure	Custom distribution → universal addressing (URLs)	Standardized document format (HTML)	Eliminated per-document distribution decisions; dissolved publishing to upload	All internet users
~2000	Search engines	Per-query information location	Manual library search → algorithmic retrieval	Minimal query encoding (keywords)	Reduced information-finding from hours to seconds; dissolved source location	All internet users
~2010	Smartphones	Per-task device selection	Multiple devices → one convergent device	Unified interface	Eliminated device-switching ops; dissolved tool selection for communication, navigation, reference	Billions of users globally

Era	Innovation	What It Dissolved	Dissolution Type	Channel Cost Change	Processing Cost Change	Population Affected
~2020	LLMs	Per-domain expert consultation for knowledge tasks	Domain-specific consultation → general query	Natural language query (minimal encoding)	Reduced domain-entry cost; partial dissolution infrastructure for arbitrary domains	Growing global population

Table N: Measurement Protocols

Term	Proxy Measurement	Direct Measurement	Tools	Validity	Limitations
Hp(A, en-code)	Composition time; revision count; draft-to-final word count change	Direct op counting from think-aloud protocol during composition	Screen recording; keystroke logging; think-aloud transcription; revision tracking	Moderate; composition time conflates encoding ops with editing and formatting	Cannot distinguish encoding ops from perfectionism or distraction; think-aloud may alter encoding process
Hs(channel)	Word count × bits/word; character count × bits/character; message size in bytes	Exact: Shannon entropy of message given source model	Standard compression ratio tools; information-theoretic analysis	High; well-established measurement	Doesn't capture receiver-relevant structure (same Hs for clear vs confusing word order)

Term	Proxy Measurement	Direct Measurement	Tools	Validity	Limitations
Hp(B, de-code)	Reading time; comprehension time; time to first correct action	Direct op counting from think-aloud during reading; eye tracking fixation patterns	Eye tracking; think-aloud protocol; comprehension testing; task performance timing	Moderate to high for reading time; lower for think-aloud (alters process)	Individual variation in reading speed conflates with processing entropy; eye tracking measures attention allocation not necessarily processing
Dissolution differential $\Delta(A,B)$	Difference in task completion time for same content between sender and receiver	Per-token decompression cost difference measured via eye tracking fixation duration on each term	Eye tracking with term-level fixation analysis; per-term comprehension testing	Moderate; fixation duration correlates with decompression cost	Fixation also reflects interest, surprise, and visual salience; not purely processing cost

Term	Proxy Measurement	Direct Measurement	Tools	Validity	Limitations
Dissolution of efficiency η	A/B test: reading time with vs without specific explanatory word/passage	Op count reduction measured via think-aloud with vs without passage	Controlled experiment; matched subjects; think-aloud or comprehension comparison	Moderate; requires careful experimental design	Per-word efficiency hard to isolate; words interact with surrounding context
Codebook alignment	Post-communication comprehension comparison: sender intent vs receiver understanding	Per-token referent elicitation from both sender and receiver independently	Interview; card sort; concept mapping; definition matching	Moderate to high for explicit tokens; low for implicit/contextual tokens	Explicit testing may create alignment that wouldn't exist in natural communication
Document quality Q	Composite: completion rate $\times$ comprehension score / (reading time $\times$ document length)	Formal: content transmitted / total cost per Table A definition	Web analytics + comprehension testing + reader surveys	Moderate; composite captures multiple dimensions	Quality definition is reader-population-relative; must specify population for meaningful measurement

Term	Proxy Measurement	Direct Measurement	Tools	Validity	Limitations
Teaching effectiveness E	Pre/post assessment score change / teaching time	Dissolution curve measurement: per-concept op count before and after teaching	Pre/post testing; timed problem solving; think-aloud protocol	High for specific concept dissolution; lower for general domain competence	Assessment may measure recall rather than dissolution; short-term gains may not persist

Table O: Specification Summary

Metric	Count
Formal definitions	17
Cost equation terms	3 (sender encoding, channel, receiver decoding)
Communication scenarios analyzed	8
Dissolution differential examples	8 communication types
Dissolution infrastructure types	10
Heterogeneous audience strategies	8
Documentation quality metrics	8
Teaching interaction phases	6
API design patterns analyzed	8
Compression ratio examples	10 tokens across processor types
Codebook alignment levels	5
Layered encoding implementations	8 media types
Cross-domain applications	9
Civilization dissolution infrastructure examples	11
Measurement protocols	8
Open problems	6

Key Equations Summary

Total communication cost:

$$\text{Cost}(A \rightarrow B) = \text{Hp}(A, \text{encode}) + \text{Hs}(\text{channel}) + \text{Hp}(B, \text{decode})$$

Compression function:

$$C: \text{referent_space} \rightarrow \text{token}; C^{-1}: (\text{token}, \text{context}) \rightarrow \text{referent}$$

Compression ratio:

$$\text{ratio}(\text{token}, \text{processor}) = |\{\text{referents decompressible across all contexts}\}|$$

Decompression cost:

$$\text{decomp}(\text{token}, \text{processor}, \text{context}) = \text{Hp}(\text{processor}, \text{decode}(\text{token}, \text{context}))$$

Dissolution differential:

$$\Delta(A, B, \text{tokens}) = \sum_t [\text{Hp}(B, \text{decode}(t)) - \text{Hp}(A, \text{decode}(t))]$$

Explanatory redundancy:

$$\text{redundancy}(\text{message}) = \text{Hs}(\text{message}) - \text{Hs}(\text{minimal_encoding})$$

Dissolution efficiency:

$$\eta(\text{word}, \text{receiver}) = -\Delta\text{Hp}(B, \text{decode}) / \Delta\text{Hs}$$

Audience-weighted dissolution efficiency:

$$\eta_{\text{audience}}(\text{word}) = \sum_i \max(0, -\Delta\text{Hp}(B_i, \text{decode})) / \Delta\text{Hs}$$

Optimal encoding:

$$\text{encoding}^* = \text{argmin}_e \{ \text{Hp}(A, \text{encode}(e)) + \text{Hs}(e) + \text{Hp}(B, \text{decode}(e)) \}$$

Heterogeneous audience cost:

$$\text{Cost}_{\text{total}} = \text{Hp}(A, \text{encode}) + \text{Hs} + \sum_{i=1}^N \text{Hp}(B_i, \text{decode})$$

Codebook alignment:

$$\text{alignment}(A, B, \text{token}) = |\text{R}_A \cap \text{R}_B| / |\text{R}_A \cup \text{R}_B|$$

Documentation quality:

$$Q(\text{doc}, \text{readers}) = \text{content_transmitted} / [\text{Hs}(\text{doc}) + \sum_i \text{Hp}(\text{reader}_i, \text{decode}(\text{doc}))]$$

Teaching effectiveness:

$$E(\text{teacher}, \text{student}, t) = -d\text{Hp}(\text{student}, \text{domain}) / dt_{\text{teaching}}$$

API quality:

$$Q(\text{API}, \text{consumers}) = \text{functionality_accessed} / \sum_i \sum_j \text{Hp}(\text{consumer}_i, \text{invoke}(\text{call}_j)) \times \text{freq}(\text{call}_j)$$

Fundamental inequality (from prior work, referenced throughout):

$$\sum \text{ops} \times \bar{d} \leq N$$

HOWL-MATH-19-2026 Appendix. The Mathematics of Processing-Aware Communication: The Three Costs of Every Message.

[[HOWL-MATH-19-2026](#)] The Mathematics of Processing-Aware Communication. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-19-2026>

[[HOWL-INFO-11-2026](#)] The Relationship of Zero, One, and Infinity in Information Processing. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-11-2026>

[[HOWL-INFO-12-2026](#)] Information Processing Requires Reduction to Cardinality One. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-12-2026>

[[HOWL-INFO-13-2026](#)] The Six States of Information. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO-13-2026>

[[HOWL-MATH-14-2026](#)] A Mathematical Theory of Processing. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-14-2026>

[[HOWL-MATH-15-2026](#)] A Measurement Theory of Processing. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-15-2026>

[[HOWL-MATH-16-2026](#)] The Geometry of Dissolution and Fragility. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-16-2026>

[[HOWL-MATH-17-2026](#)] Processing Entropy as a Metric Space. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-17-2026>

[[HOWL-MATH-18-2026](#)] Concurrency Tax from System Structure. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-18-2026>